

2026-09-07 更新：Gate 训练数据构造新方案 (query 形式统一 + wavefront 真实负样本 + 对比学习就绪)

本文档整理下一版 gate (static/pooled, `gate/fit_lr_gate_pooled.py` 这条生产线) 训练数据构造的完整新方案, 替代现在 `traces/construct_balanced_traces.py` 的构造逻辑。只改 stage 2 (trace 构造), stage 3 (隐藏状态 提取)、stage 4 (PCA+LR 拟合) 不动。

零、之前的数据是怎么构造的

`traces/construct_balanced_traces.py` (现在生产在用的 stage 2) 是纯静态、确定性的构造方式, 不跑模型、不做真实检索, 两种 trace 类型都是直接从 GT 拆解 + gold 证据表里查出来拼出来的:

- **gold trace** (`trace_type=correct`): K 跳全部用 gold 证据, `wrong_hops=[]`。子问题文本直接用 GT 拆解 出来的原始句子拼进 `reasoning_trace`, 没有调用 `expand_hop_template`——`[Answer N]` 占位符大约 49% 的情况下没有被替换成真实上一跳答案, 就是字面的 `[Answer N]` 文本。
- **counterfactual trace** (`trace_type=error`): 挑一跳 h, 把这一跳的证据换成 cosine-top-k 里排名靠前的一个非 gold 段落, `wrong_hops=[h]`; 除了这一跳之外的其余所有跳, 包括 h 之后的跳, 都照抄 gold 证据、gold 答案——也就是说不管 h 这一跳选错了会不会导致下游子问题的 `[Answer N]` 展开出错、下游检索会不会 因此检索到不同的候选, 这条链路完全没有被模拟, h 之后的部分永远是"未受污染的" gold。
- 两种 trace 各自按 (K, hop) 分桶做 up/downsample 配平, 写进 `merged/`, stage 3/4 只读这个目录。

这套构造方式的好处是简单、快 (不用调生成模型/真实检索), 但也正是"一、动机"里三个问题的根源: query 形式不统一、负样本不真实、没有对比学习需要的结构化 pair 数据。

现在这套方案的好处

- **检索 query 形式统一**: 不管是训练还是检索/reader, 子问题一律用 `expand_hop_template` 展开过的文本, 不再需要 `_rawprefix.py` 那种维护两条平行文本轨道的补丁。
- **负样本是真实级联传播出来的, 不是人为只改一跳**: 某一跳出错之后, 后续跳的检索 query、检索结果、生成 的短答案全部是真实推理续推出来的结果, 实测能看到错误确实会顺着 `[Answer N]` 污染下游子问题 (比如 hop3 生成了 "NA", hop4 的展开 query 就真的带出了 "...NA?" 这种坏问法) ——这正是 gate 在真实推理时需要 识别、介入的场景, 老数据里完全没有这类样本。
- **一个 case 产出多条互补的 trace, 覆盖的场景比"一个 case 一条 gold + 若干条单跳错误"丰富得多**: 既有 完全真实的 baseline 轨迹 (recipe 2, 标了 `is_natural_seed`), 也有"前 a 跳保证干净、之后完全真实续推"的一系列中间状态 (recipe 4), 还有"某一跳故意出错、之后完全真实续推"的对照 (recipe 3), 而且都经过跨 recipe 的去重, 不会有大量冗余重复的样本。
- **每条 trace 自带最终答案的 EM/F1**, 能同时看出"检索/证据链对不对"和"最终答案对不对"这两件事之间的关系 (检索出错不一定最终答案就错, 反之亦然), 老数据完全没有这个信号。
- **对比学习需要的 pair 数据是构造时顺手生成的, 不用等以后再回头补**: 每个 pair 共享同一个 `h_prev`, 分别接正确/错误证据各算一次隐藏状态, 直接对得上 `model.py::ListwiseGateModel` 现有的 `h_prev/h_after/delta` 特征结构。

- 隐藏状态存储格式跟现有 **pipeline** 完全兼容（通过 `export_to_legacy_hidden_states.py` 适配层），`gate/fit_lr_gate_pooled.py` 零改动就能直接读，stage 3/4 不用碰。

一、动机：现在的构造方式有三个问题

1. 检索用的 **query** 形式和 **gate** 训练用的文本形式不一致。BGE 检索的 query 是 `expand_hop_template` 展开过的（`[Answer N]` 替换成上一跳短答案）；但 `construct_balanced_traces.py` 组装 `reasoning_trace` 字段时，**没有调用 `expand_hop_template`**，直接拿 GT 原始子问题塞进去——大约 49% 的 hop 文本里 `[Answer N]` 是没替换的字面占位符（这是 `run_retrieval_exp_wavefront_gate_v3_rawprefix.py` docstring 里核实过的 训练数据事实）。现在生产用的 `_rawprefix.py` 是靠维护两条平行文本轨道（展开过的给检索/reader，原始的给 gate 打分）来绕开这个问题，不是从根上统一。
2. 负样本“只错一跳，其余全干净”不真实。现在的 counterfactual trace：只把 hop h 的证据换成错的，hop h 之后所有跳依然用 gold 证据、gold 短答案展开——但如果 hop h 真的选错了，抽出来的短答案大概率也是错的，会顺着 `[Answer N]` 传染给下游子问题，下游的检索也会因为 query 变了而检索到不同的（可能也错的）候选。现在的构造方式完全没有模拟这个传播过程。
3. 想为以后的对比学习（**contrastive learning**）预先准备好数据，不想等以后要用的时候再回头重新构造一遍。

二、整体思路：复用 wavefront 推理骨架，不用 gate，把 gold 当“种子”

不是从零写一个新的构造脚本，而是复用已经跑通的、hop-batched 的 wavefront 推理循环（`train_online_wavefront.py/run_wavefront_online_gate.py` 那套：真实检索 → 选一个候选 → 真实调 Llama 生成短答案 → 传播到下一跳的 query → 重复，整批样本一起批处理），把“用 gate 打分选 top1”换成“用纯 **cosine top-1** 选”（即 baseline 方法），在 train 集上跑一遍。每条样本自己的 gold trace（gold 证据 + gold 短答案）全程都在手边，当“种子”用：

- 每一跳选完之后，跟 gold 比一下 → 就是这一跳的标签（选对 = False / 不该介入，选错 = True / 该介入），是**事后比对出来的**，不是构造前预先设计好的传播规则；
- 选错的地方，因为 gold 证据也在手上 → 顺手记一条**对比学习用的 pair**（真实选中的错误证据 vs gold 正确 证据，同一个上下文）；
- 需要“强制某几跳走 gold”的时候，gold 就是这个“种子”，从种子接回真实推理继续往下走。

case 覆盖范围：跟旧 pipeline 对齐，MuSiQue train 除了原始 GT 拆解覆盖的 19,938 个 case，还加上针对 K=3/4 做的 LLM 复述增强 case（子问题换一种问法，经过校验），合计 24,070 个 case——K=3/4 这两个桶里超过 40% 的 case 只存在于增强部分，如果不算进来，长链条（更容易出现级联传播）的样本会明显偏少。

检索侧的 query 形式：cosine 检索遵循 BGE（`bge-*-en-v1.5`）官方对非对称检索（短 query 查长 passage）的用法约定——query 侧统一加上检索指令前缀，passage/段落侧不加，两边不对称编码；本来这套 pipeline（包括旧的 `construct_balanced_traces.py`）里都没加这个前缀，这次一并修正。

三、四种数据来源（recipe），本质是同一个“强制到第几跳”参数的不同取值

一个 K 跳的 case，每一跳原则上要么"强制走 gold"，要么"强制选一个错的"，要么"完全交给真实检索/生成"。四种 recipe 就是这三种操作的四种组合方式，都要跑，不是互斥的：

recipe	做法
1. 纯 gold（前缀长度 a=K）	全程 gold 证据 + gold 短答案， <code>expand_hop_template</code> 统一展开（解决动 机 1 的问题）。确定性构造，不调生成模型，永远全对。
2. 真实 baseline 逐跳推理（前缀长度 a=0，"种子"）	每一跳都真实检索、真实调 Llama 生成短答案、真实传播——不管最后走没走 对，整条如实记录，事后跟 gold 比对打标签。
3. 单跳强制出错	挑一跳（h=1..K，每个位置各生成一条变体），强制换成 cosine 排序里相似度 最高的非 gold 候选；这一跳之前是真实检索的自然结果，之后完全交给真实检 索/生成续推（不会被强制拉回 gold，也不会被强制成更多的错）。
4. 干净前缀扫描（前缀长度 a=1..K-1）	强制前 a 跳走 gold（子问题展开用 gold 短答案），从第 a+1 跳开始完全交给 真实检索/生成续推。a=0 和 a=K 分别就是 recipe 2 和 recipe 1，中间的 a 值就是这个 recipe。

关键设计原则：强制"变错"可以发生在任何位置，但强制"变对（gold）"只能发生在链条最前面、还没有出过错的那一段。一旦某一跳被强制或自然地选错了，后面的跳绝不会再被强制拉回 gold——因为真实推理里不存在"上一跳已经错了，后面又有人把 gold 证据塞回来"这种情况；如果允许这样构造，等于在教 gate"哪怕上下文已经不对劲，只要证据技术 上是对的就不用管"，这是一个错误的信号，不应该出现在训练数据里。recipe 3/4 之后的部分永远是"完全放开、如实 记录"，不会有第三种"强制回正确"的操作。

不管 recipe 2 本身自己走没走通，recipe 3 都要对每个 case 的每个位置生成一遍（哪怕这道题 baseline 自己就能 走全对，也要看"故意在某一跳出错、其余交给真实检索"会发生什么，这本身是有价值的训练信号）；recipe 4 同理，对每个 case 的每个中间前缀长度都生成一遍，不只是 recipe 2 失败的 case 才补。

四、去重规则

recipe 1 / 2 去重：recipe 2 如果自己跑下来全部 K 跳都跟 gold 对上了，这条真实轨迹本身就是一条高质量 正样本（比 recipe 1 更真实，因为短答案是 Llama 真实生成的文本，不是数据集里查表拿到的 canonical gold 文本）——这种情况下不再额外造 recipe 1，两者信息高度冗余。只有 recipe 2 没有全对的 case，才补一条 recipe 1（保证每个 case 至少有一条 100% 可靠的干净 trace，不受 baseline 检索质量影响）。

跨 recipe 的签名去重：recipe 3/4 的某个变体，可能巧合跟 recipe 2（或者互相之间）生成出完全一样的证据 序列——比如 recipe 4 强制前 a 跳走 gold，如果 recipe 2 本身在那几跳自然就选对了，前缀部分就跟 recipe 2 一 模一样，续推的尾部如果也巧合一致，整条就是重复的；recipe 3 强制某一跳"选错"，如果 recipe 2 自己在那一跳 本来就自然选错、而且选的是同一个候选，也会完全重复。做法：以"每一跳的 `committed_idx` 序列"为签名，同一个 case 内后出现的重复签名直接丢弃，只保留最先生成的那条（生成顺序 recipe2→recipe1→recipe3→recipe4，天然 优先保留"真实种子"或"确定性 gold"这两种更有信息量的来源）。

五、输出三份文件，用 id 串起来

一个 case（原始样本）对应**多条 trace**（recipe 2 一条 + recipe 3 每个位置一条 + recipe 4 每个前缀长度 一条 + 可能的 recipe 1，去重之后可能更少），所以不是"一个 case 一行数据"，是"一个 case 多行

trace, 用 `case_id` 关联”。

5.1 `traces_v2/{dataset}/{split}.jsonl` —— 一行一条 trace

```
{
  "trace_id": "2hop__12345_67890__recipe2_real",
  "case_id": "2hop__12345_67890",
  "recipe": "2_real_baseline",
  "is_natural_seed": true,
  "K": 3,
  "question": "Where is the headquarters of the company founded by the
director of Film X?",
  "hops": [
    {
      "hop": 1,
      "sub_question_raw": "Who is the director of Film X?",
      "sub_question_expanded": "Who is the director of Film X?",
      "cosine_top3": [
        {"idx": 5, "title": "Film X", "score": 0.91},
        {"idx": 12, "title": "Director Y", "score": 0.77},
        {"idx": 3, "title": "Studio Z", "score": 0.65}
      ],
      "committed_idx": 5,
      "gold_idx": 5,
      "is_correct": true,
      "short_answer_generated": "John Smith",
      "hidden_state_ref": {"npz": "2hop__12345_67890__recipe2_real.npz",
"prefix_idx": 1}
    },
    {
      "hop": 2,
      "sub_question_raw": "What company did [Answer 1] found?",
      "sub_question_expanded": "What company did John Smith found?",
      "cosine_top3": [
        {"idx": 41, "title": "Zenith Corp", "score": 0.83},
        {"idx": 9, "title": "Acme Studios", "score": 0.79},
        {"idx": 17, "title": "Other Co", "score": 0.60}
      ],
      "committed_idx": 41,
      "gold_idx": 9,
      "is_correct": false,
      "short_answer_generated": "Zenith Corp",
      "pair_id": "2hop__12345_67890__recipe2_real__hop2",
      "hidden_state_ref": {"npz": "2hop__12345_67890__recipe2_real.npz",
"prefix_idx": 2}
    },
    {
      "hop": 3,
      "sub_question_raw": "Where is the headquarters of [Answer 2]
located?",
      "sub_question_expanded": "Where is the headquarters of Zenith Corp
located?",
```

```

    "cosine_top3": [...],
    "committed_idx": 88,
    "gold_idx": 20,
    "is_correct": false,
    "short_answer_generated": "unknown",
    "pair_id": "2hop__12345_67890__recipe2_real_hop3",
    "hidden_state_ref": {"npz": "2hop__12345_67890__recipe2_real.npz",
"prefix_idx": 3}
  },
  ],
  "final_answer_generated": "Zenith Corp",
  "final_answer_em": false,
  "final_answer_f1": 0.0
}

```

`is_natural_seed` 只有 recipe 2 这一条是 `true`，其余全部 `false`——标记“哪条是真实 cosine 自然召回出来的，哪些是人为构造的”。`final_answer_generated/final_answer_em/final_answer_f1` 是这条 **trace** 实际走完之后，把它最终的证据链（每一跳 `committed_idx` 对应的证据文本，不是 gold）喂给最终 reader（跟生产环境 同一套 prompt/同一个本地 Llama）生成答案、跟 gold 答案算出来的——不是检索层面的对错，是“这条具体轨迹最后 答对了没有”。这个字段对每条 trace 都要算（不只是 recipe 2），因为价值在于能看出“检索层面出没出错”和“最后 答案对不对”这两件事不完全同步——比如某条 trace 某一跳选错了，但最终答案因为 reader 拿到的冗余信号（子问题 + 证据原文 + 短答案）还是蒙对了；或者反过来，每一跳检索都对，但最终答案还是生成错了。

因为这条 recipe 2 没有全对（hop2/3 都错了），按第四节的重去规则，同一个 `case_id` 下面还会补一条：

```

{
  "trace_id": "2hop__12345_67890__recipe1_gold",
  "case_id": "2hop__12345_67890",
  "recipe": "1_gold_injected",
  "is_natural_seed": false,
  "K": 3,
  "hops": [ "...三跳全部 is_correct=true, 子问题用 expand_hop_template + gold
答案统一展开..." ]
}

```

以及 recipe 3 的三个变体（强制在 hop1/hop2/hop3 各出错一次）：

```

{
  "trace_id": "2hop__12345_67890__recipe3_forced_h1",
  "case_id": "2hop__12345_67890",
  "recipe": "3_forced_wrong",
  "forced_hop": 1,
  "K": 3,
  "hops": [ "...hop1 被强制换成一个高相似度非 gold 候选, hop2/3 是从这个错误状态开
始真实续推的结果..." ]
}

```

和 recipe 4 的两个变体（前缀长度 a=1、a=2）：

```
{
  "trace_id": "2hop__12345_67890__recipe4_cleanprefix_a1",
  "case_id": "2hop__12345_67890",
  "recipe": "4_clean_prefix",
  "clean_prefix_upto": 1,
  "K": 3,
  "hops": [ "...hop1 强制 gold, hop2/3 真实续推..." ]
}
```

5.2 hidden_states_v2/{dataset}/{split}/{trace_id}.npz —— 原始（未 PCA）隐藏状态，存法跟现在完全一样

文件名是 `trace_id`（不是 case id），这样一个 case 的多条 trace 各自有独立的 npz 文件，不会互相覆盖。每个 hop 的 `hidden_state_ref` 指到这个文件里第几个 prefix。**PCA 依然只在 stage 4 训练时现拟合，不预先存 PCA 后的数据**——这一点跟现在的 pipeline 一致，不用额外做什么。

5.3 pairs_v2/{dataset}/{split}.jsonl —— 一行一个对比学习用的 pair

```
{
  "pair_id": "2hop__12345_67890__recipe2_real__hop2",
  "case_id": "2hop__12345_67890",
  "trace_id": "2hop__12345_67890__recipe2_real",
  "hop": 2,
  "context_prefix": "Question: ... Step 1: Who is the director of Film X?",
  "Evidence: \"...\"",
  "sub_question": "What company did John Smith found?",
  "evidence_correct": {"idx": 9, "text": "... (gold Acme Studios 段落原文) ..."},
  "evidence_wrong": {"idx": 41, "text": "... (Zenith Corp 段落原文) ..."},
  "hidden_correct_ref": {"npz": "2hop__12345_67890__recipe2_real.npz",
  "pair_idx": 0, "which": "correct"},
  "hidden_wrong_ref": {"npz": "2hop__12345_67890__recipe2_real.npz",
  "pair_idx": 0, "which": "wrong"}
}
```

`context_prefix/sub_question` 两边共享（同一个决策点），`evidence_correct/evidence_wrong` 是这个决策点 上真实存在的一对候选。**隐藏状态需要单独多算一次**：trace 本身只记录了真实发生的那个分支（选了 wrong 的那个）的隐藏状态；pair 需要同一个 `h_prev`（hop 开始前的状态，两边共享）分别接上 `evidence_correct` 和 `evidence_wrong` 各算一次 `h_after`——正好是 `model.py::ListwiseGateModel` 已经在用的 `h_prev/h_after/delta` 结构，pair 数据天然对得上现有的特征设计，以后接对比学习或者接现有的 online listwise 特征管线都不用改格式。

六、跟现有 pipeline 的关系

- stage 3 (`hidden_states/extract_hidden_states.py`)、stage 4 (`gate/fit_lr_gate_pooled.py`) 不改——只是读取的输入从 `traces/merged/` 换成 `traces_v2/`，字段兼容 (`wrong_hops` 可以从新 schema 的 `hops` 数组直接派生：`wrong_hops = [h["hop"] for h in hops if not h["is_correct"]]`)。
- `run_retrieval_exp_wavefront_gate_v3_rawprefix.py` 维护的"两条平行文本轨道"补丁，理论上不再需要——新训练数据从构造时就统一用展开过的子问题，gate 训练和推理看到的文本形式天然一致。

这套构造方式已经在小规模数据上实现并验证：真实检索/生成的每一步（hop1 检索命中率、短答案语义正确性、跨跳错误级联传播、pair 隐藏状态提取、去重效果等）都用具体样本核对过，行为符合上述设计。全量数据构造完成后，下一步是重跑 stage 3/4（隐藏状态提取 + gate pooled 拟合），跟现有 gate 的 `recall@1` vs `recall@3` 数字 做对比，检验这次重构是否真的改善了这个 gap——这是这次改动最终想解决的问题。